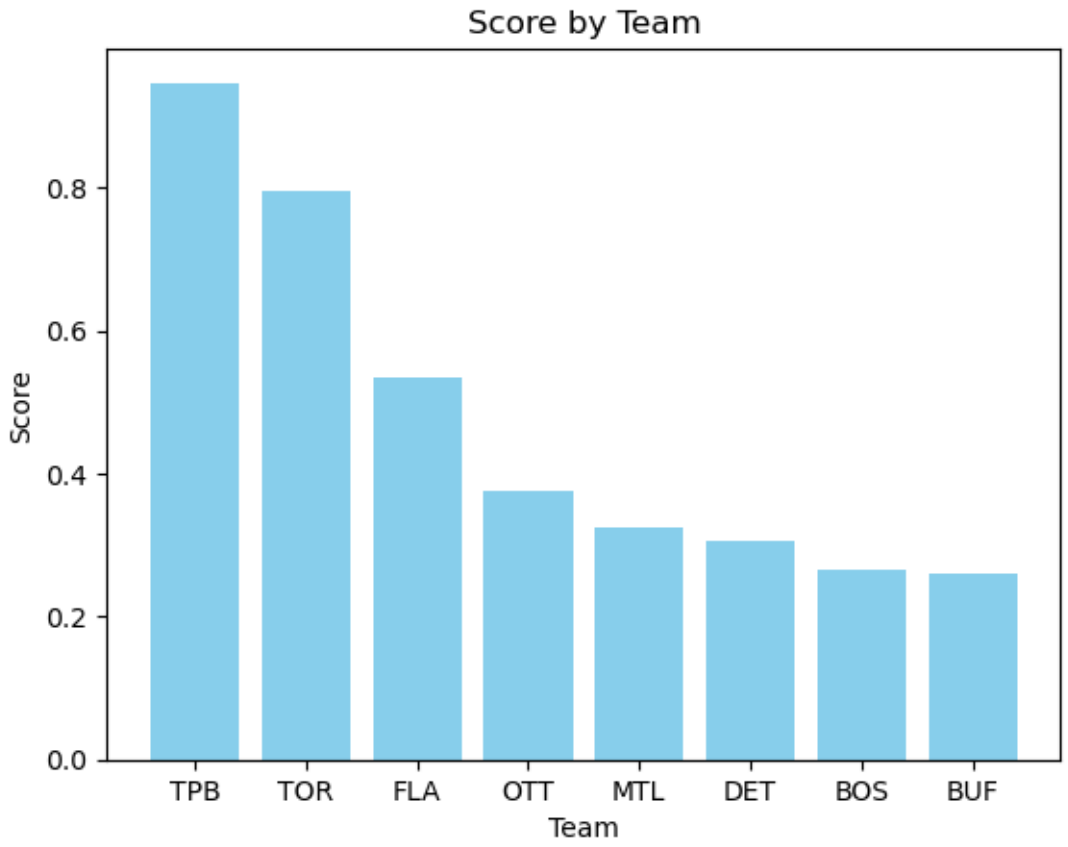
standings

January 29, 2026

[7]:
```
# With differential

import numpy as np
import matplotlib.pyplot as plt

teams = np.array(["TOR", "TPB", "FLA", "OTT", "MTL", "DET", "BUF", "BOS"])
differential = np.array([37, 75, 29, 9, -20, -21, -20, -50])
points_of_best_player = np.array([104, 121, 81, 79, 89, 80, 72, 106])
goals_of_best_scorer = np.array([45, 42, 39, 29, 37, 39, 44, 43])
best_goalie_svp = np.array([.926, .921, .906, .910, .902, .901, .887, .893])

def min_max_normalize(data):
    return (data - np.min(data)) / (np.max(data) - np.min(data))

# Normalize the data
differential_norm = min_max_normalize(differential)
points_of_best_player_norm = min_max_normalize(points_of_best_player)
goals_of_best_scorer_norm = min_max_normalize(goals_of_best_scorer)
best_goalie_svp_norm = min_max_normalize(best_goalie_svp)

weights = np.array([0.5, 0.15, 0.15, 0.2])
pred = np.zeros(8)

def neural_network(input, weights):
    return input.dot(weights)

# Store team scores
team_scores = []

for i in range(8):
    input = np.array([differential_norm[i], points_of_best_player_norm[i],␣
 ↪goals_of_best_scorer_norm[i], best_goalie_svp_norm[i]])
    score = neural_network(input, weights)

    team_scores.append((teams[i], score))

# Sort the list
```

```
team_scores.sort(key=lambda x: x[1], reverse=True)

# Prepare data for the bar chart
teams_sorted = [team for team, score in team_scores]
scores_sorted = [score for team, score in team_scores]

# Bar chart
plt.bar(teams_sorted, scores_sorted, color='skyblue')

plt.title("Score by Team")
plt.xlabel('Team')
plt.ylabel('Score')

# Show plot
plt.show()
```

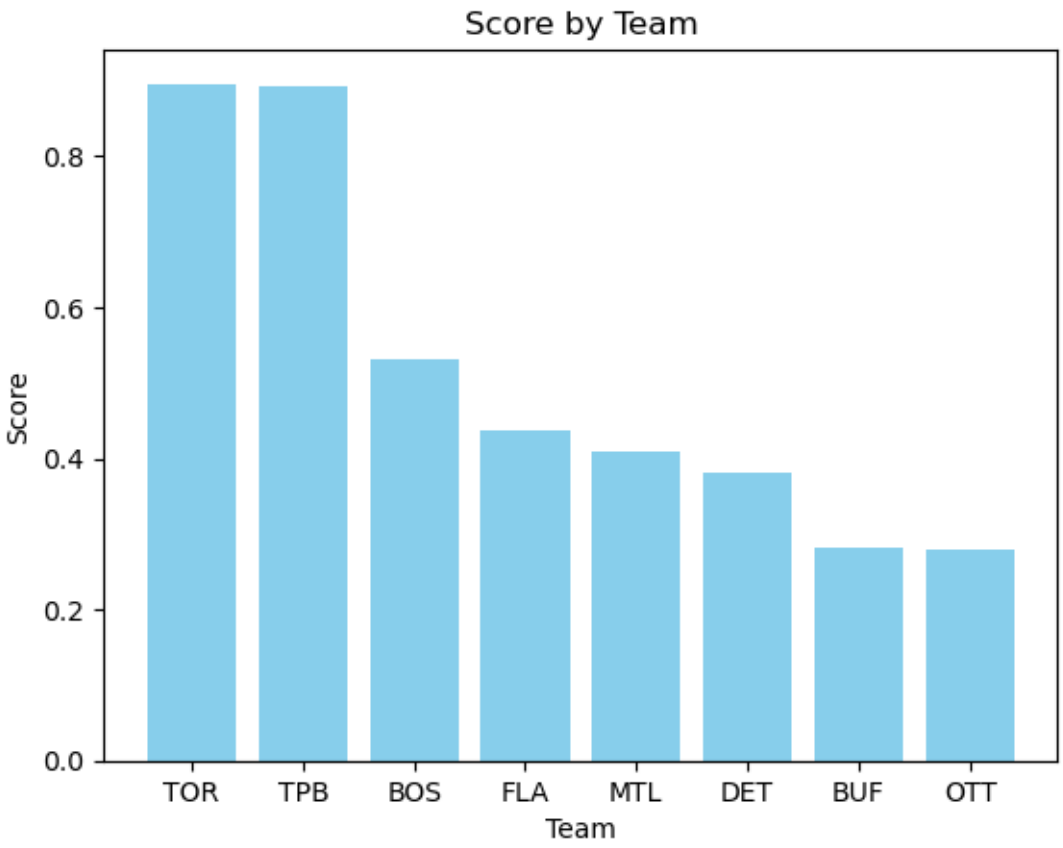


[11]:
```
# This shows how a team's best player's individual effort impacted their team
↪(differential is of 0 weight)
```

```
# Considering the goaler is the most important player on the ice and that their␣
 ↪performance is impacted by
# play in front of them it has a bigger percentage

# Data Initialization
teams = np.array(["TOR", "TPB", "FLA", "OTT", "MTL", "DET", "BUF", "BOS"])
differential = np.array([37, 75, 29, 9, -20, -21, -20, -50])
points_of_best_player = np.array([104, 121, 81, 79, 89, 80, 72, 106])
goals_of_best_scorer = np.array([45, 42, 39, 29, 37, 39, 44, 43])
best_goalie_svp = np.array([.926, .921, .906, .910, .902, .901, .887, .893])

# Min-Max Normalization Function
def min_max_normalize(data):
    return (data - np.min(data)) / (np.max(data) - np.min(data))

# Normalize the data
differential_norm = min_max_normalize(differential)
points_of_best_player_norm = min_max_normalize(points_of_best_player)
goals_of_best_scorer_norm = min_max_normalize(goals_of_best_scorer)
best_goalie_svp_norm = min_max_normalize(best_goalie_svp)

weights = np.array([0.0, 0.3, 0.3, 0.4])
pred = np.zeros(8)

def neural_network(input, weights):
    return input.dot(weights)

# Store team scores
team_scores = []

for i in range(8):
    input = np.array([differential_norm[i], points_of_best_player_norm[i],␣
 ↪goals_of_best_scorer_norm[i], best_goalie_svp_norm[i]])
    score = neural_network(input, weights)

    team_scores.append((teams[i], score))

# Sort the list of tuples based on scores in descending order
team_scores.sort(key=lambda x: x[1], reverse=True)

# Print the sorted team scores
for team, score in team_scores:
    print(team, score)

# Sort the list
team_scores.sort(key=lambda x: x[1], reverse=True)
```

```
# Prepare data for the bar chart
teams_sorted = [team for team, score in team_scores]
scores_sorted = [score for team, score in team_scores]

# Bar chart
plt.bar(teams_sorted, scores_sorted, color='skyblue')

plt.title("Score by Team")
plt.xlabel('Team')
plt.ylabel('Score')

# Show plot
plt.show()
```

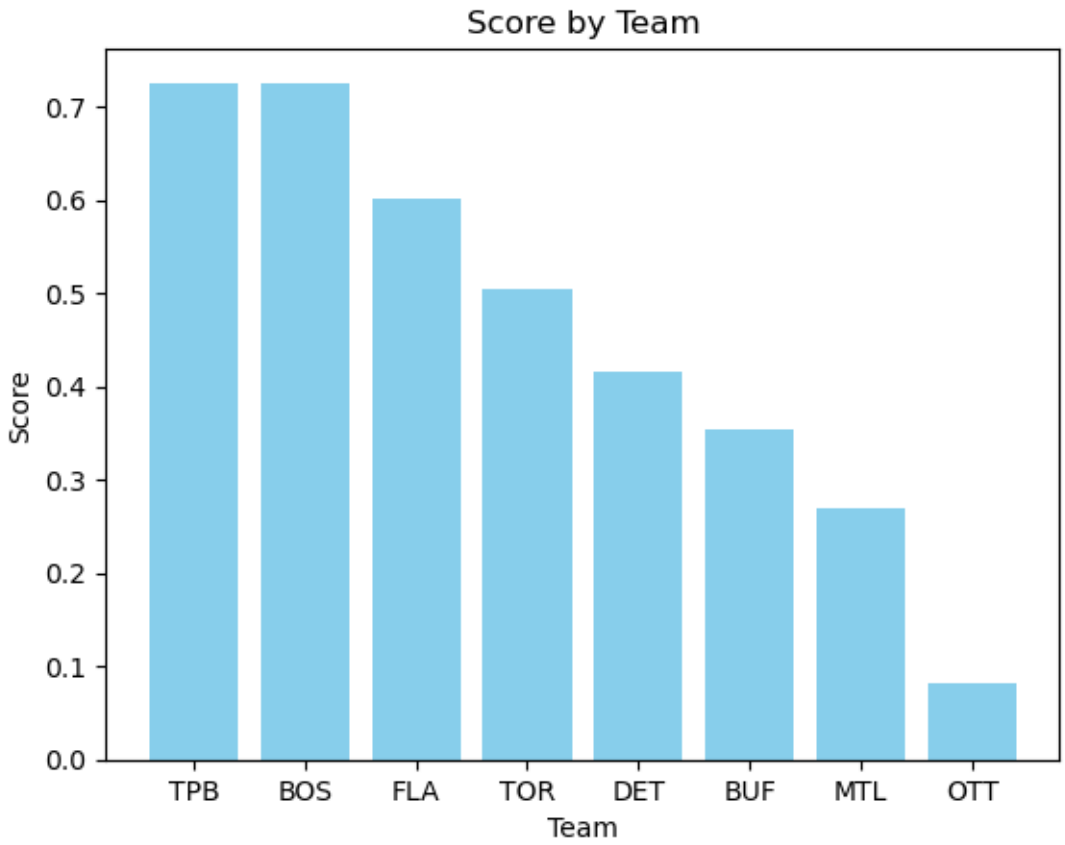
```
TOR 0.8959183673469387
TPB 0.8924679487179488
BOS 0.532201726844584
FLA 0.43747383568812137
MTL 0.4079277864992151
DET 0.3800693354264783
BUF 0.28125
OTT 0.27875457875457876
```



[16]:
```
# This shows how a team's best player's individual effort impacted their team␣
 ↪(differential is of 0 weight)
# Considering the goaler is the most important player on the ice and that their␣
 ↪performance is impacted by
# play in front of them it has a bigger percentage

# Data Initialization (05 January 2026)
teams = np.array(["TOR", "TPB", "FLA", "OTT", "MTL", "DET", "BUF", "BOS"])
points_of_best_player = np.array([41, 59, 46, 43, 46, 44, 37, 46])
goals_of_best_scorer = np.array([20, 20, 23, 19, 20, 22, 20, 25])
best_goalie_svp = np.array([.914, .912, .904, .881, .889, .895, .906, .906])

# Min-Max Normalization Function
def min_max_normalize(data):
    return (data - np.min(data)) / (np.max(data) - np.min(data))

# Normalize the data
differential_norm = min_max_normalize(differential)
points_of_best_player_norm = min_max_normalize(points_of_best_player)
goals_of_best_scorer_norm = min_max_normalize(goals_of_best_scorer)
best_goalie_svp_norm = min_max_normalize(best_goalie_svp)

weights = np.array([0.3, 0.3, 0.4])
pred = np.zeros(8)

def neural_network(input, weights):
    return input.dot(weights)

# Store team scores
team_scores = []

for i in range(8):
    input = np.array([points_of_best_player_norm[i],␣
 ↪goals_of_best_scorer_norm[i], best_goalie_svp_norm[i]])
    score = neural_network(input, weights)

    team_scores.append((teams[i], score))

# Sort the list of tuples based on scores in descending order
team_scores.sort(key=lambda x: x[1], reverse=True)

# Print the sorted team scores
for team, score in team_scores:
    print(team, score)
```

```
# Sort the list
team_scores.sort(key=lambda x: x[1], reverse=True)

# Prepare data for the bar chart
teams_sorted = [team for team, score in team_scores]
scores_sorted = [score for team, score in team_scores]

# Bar chart
plt.bar(teams_sorted, scores_sorted, color='skyblue')

plt.title("Score by Team")

# Setting the X and Y labels
plt.xlabel('Team')
plt.ylabel('Score')

# Show plot
plt.show()
```

```
TPB 0.7257575757575758
BOS 0.7257575757575758
FLA 0.6015151515151516
TOR 0.5045454545454545
DET 0.41515151515151516
BUF 0.353030303030303
MTL 0.2696969696969697
OTT 0.0818181818181818
```



[18]:
```
# This shows how a team's best player's individual effort impacted their team␣
 ↪(differential is of 0 weight)
# Considering the goaler is the most important player on the ice and that their␣
 ↪performance is impacted by
# play in front of them it has a bigger percentage

# Data Initialization (TOR projected ideal season)
teams = np.array(["TOR", "TPB", "FLA", "OTT", "MTL", "DET", "BUF", "BOS"])
differential = np.array([37, 75, 29, 9, -20, -21, -20, -50])
points_of_best_player = np.array([90, 121, 81, 79, 89, 80, 72, 106])
goals_of_best_scorer = np.array([40, 42, 39, 29, 37, 39, 44, 43])
best_goalie_svp = np.array([.910, .921, .906, .910, .902, .901, .887, .893])

# Min-Max Normalization Function
def min_max_normalize(data):
    return (data - np.min(data)) / (np.max(data) - np.min(data))

# Normalize the data
differential_norm = min_max_normalize(differential)
```

```
points_of_best_player_norm = min_max_normalize(points_of_best_player)
goals_of_best_scorer_norm = min_max_normalize(goals_of_best_scorer)
best_goalie_svp_norm = min_max_normalize(best_goalie_svp)

weights = np.array([0.0, 0.3, 0.3, 0.4])
pred = np.zeros(8)

def neural_network(input, weights):
    return input.dot(weights)

# Store team scores
team_scores = []

for i in range(8):
    input = np.array([differential_norm[i], points_of_best_player_norm[i],␣
 ↪goals_of_best_scorer_norm[i], best_goalie_svp_norm[i]])
    score = neural_network(input, weights)

    team_scores.append((teams[i], score))

# Sort the list of tuples based on scores in descending order
team_scores.sort(key=lambda x: x[1], reverse=True)

# Print the sorted team scores
for team, score in team_scores:
    print(team, score)

# Sort the list
team_scores.sort(key=lambda x: x[1], reverse=True)

# Prepare data for the bar chart
teams_sorted = [team for team, score in team_scores]
scores_sorted = [score for team, score in team_scores]

# Bar chart
plt.bar(teams_sorted, scores_sorted, color='skyblue')

plt.title("Score by Team")

# Setting the X and Y labels
plt.xlabel('Team')
plt.ylabel('Score')

# Show plot
plt.show()
```

```
TPB 0.9600000000000001
BOS 0.55875150060024
```

```
TOR 0.48314525810324127
FLA 0.47863145258103246
MTL 0.44055222088835533
DET 0.41368547418967583
OTT 0.31344537815126056
BUF 0.3
```